

REVISÃO DE LITERATURA: CONTAINERS E EFICIÊNCIA DE PIPELINES DE DADOS

1 TEMA CENTRAL

O presente trabalho tem como tema central a infraestrutura de pipelines e orquestração em Engenharia de Dados, com ênfase no impacto do uso de containers na eficiência, escalabilidade e confiabilidade dos processos de tratamento e movimentação de dados em ambientes modernos de grande escala.

2 PERGUNTAS DE PESQUISA IDENTIFICADAS

A partir do tema central, foram formuladas três perguntas de pesquisa, listadas a seguir:

1. O uso de containers afeta a eficiência das pipelines de dados em ambientes de engenharia de dados de grande escala?
2. Existe grande impacto das pipelines de dados na qualidade e escalabilidade dos dados em empresas orientadas a dados?
3. Como a orquestração de pipelines em infraestruturas de nuvem altera o uso de recursos computacionais?

Após avaliação das três perguntas quanto à especificidade, mensurabilidade e disponibilidade de literatura acadêmica, a Pergunta 1 foi selecionada como foco principal do trabalho, por apresentar maior precisão técnica e maior volume de estudos disponíveis no Consensus.

3 PERGUNTA DE PESQUISA SELECIONADA

O uso de containers afeta a eficiência das pipelines de dados em ambientes de engenharia de dados de grande escala?

4 CONSENSO ACADÊMICO

A consulta realizada na plataforma Consensus, ferramenta de busca acadêmica baseada em inteligência artificial, revelou um consenso positivo e bem estabelecido a respeito do tema. A literatura acadêmica indica consistentemente que containers aumentam a eficiência de pipelines de dados, especialmente em cenários distribuídos e de grande volume. Os estudos apontam ganhos expressivos em throughput, latência, escalabilidade e reprodutibilidade, com overhead de performance classificado como pequeno ou desprezível para tarefas de média e alta duração. A única ressalva recorrente diz respeito ao custo inicial para configurar o ambiente e as instâncias dos containers.

Entre os achados mais relevantes identificados pelo Consensus, destacam-se:

- Pipelines genômicas containerizadas com Docker apresentaram impacto mínimo no tempo de execução, com o pequeno custo sendo amplamente compensado pela facilidade de empacotamento e reprodutibilidade do ambiente (DI TOMMASO et al., 2015).
- Pipelines IoT containerizadas em Kubernetes com autoscaling alcançaram aumento de throughput entre 1,31x e 2,4x, com redução de latência entre 32x e 80x frente a alocações estáticas de recursos (AURANGZAIB et al., 2022).
- Um modelo de pipeline encapsulado em containers reduziu tempos de resposta entre 43% e 91% em comparação a pipelines em memória e outras engines (SANTIAGO DURAN et al., 2020).
- Em ambientes de ETL na área da saúde, a combinação de PySpark e Docker trouxe ganhos relevantes de tempo de processamento, uso de recursos e escalabilidade (SOLTANMOHAMMADI; HIKMET, 2024).

5 CONSIDERAÇÕES MAIS IMPORTANTES A PESQUISAR

Com base no consenso acadêmico e nas lacunas identificadas na literatura, as seguintes dimensões se mostram centrais para o aprofundamento da pesquisa:

Dimensão	O que investigar
Performance	Impacto real no tempo de execução e throughput em diferentes cargas e tipos de dados
Escalabilidade	Como o autoscaling via Kubernetes responde a variações súbitas de demanda em pipelines de dados
Reprodutibilidade	Consistência de resultados entre ambientes de desenvolvimento, teste e produção
Trade offs operacionais	Overhead de startup de containers e curva de aprendizado para equipes de engenharia
Contexto setorial	Variações de resultado entre setores: saúde, IoT, machine learning e analytics corporativo
Ferramentas de orquestração	Comparação entre Apache Airflow, Prefect e Dagster em ambientes containerizados

6 JUSTIFICATIVA

A crescente adoção de arquiteturas orientadas a dados nas organizações torna a eficiência das pipelines de dados um fator crítico de competitividade e inovação. Nesse contexto, a containerização emerge como uma das principais estratégias de infraestrutura em engenharia de dados, impactando diretamente a velocidade de processamento, a confiabilidade operacional e a capacidade de escalonamento dos sistemas.

Estudos recentes demonstram ganhos expressivos. Pipelines de dados em tempo real para Internet das Coisas, containerizadas em Kubernetes, alcançaram aumento de throughput entre 1,31x e 2,4x, com redução de latência de até 80x frente a alocações estáticas de recursos (AURANGZAIB et al., 2022). Em ambientes de extração, transformação e carga na área da saúde, a combinação de PySpark e Docker resultou em ganhos significativos de tempo de processamento e uso de recursos (SOLTANMOHAMMADI; HIKMET, 2024).

Além da performance bruta, containers oferecem benefícios operacionais relevantes para equipes de engenharia de dados: portabilidade entre ambientes multi cloud, isolamento de dependências, modularidade de estágios de pipeline e recuperação automática de falhas via orquestradores como o Kubernetes, características fundamentais para pipelines confiáveis em produção (GANDHARI, 2025; TIWARI, 2025).

Compreender como e sob quais condições esses ganhos se materializam é, portanto, essencial para orientar decisões estratégicas de infraestrutura em times de engenharia de dados modernos, especialmente diante da expansão acelerada de volumes de dados e da diversificação das fontes e destinos de informação nas organizações contemporâneas.

7 ESTRATÉGIA DE MAPEAMENTO DAS FONTES

Para a seleção das fontes utilizei a técnica de snowballing, com apoio de ferramentas de mapeamento de citações, o Litmaps. Parti de artigos origem de alta relevância e impacto e, a partir deles, explorei o grafo de citações em dois sentidos:

- **Para trás, retrospectivo:** identificação das obras mais antigas e mais citadas que fundamentam o tema, os chamados clássicos.
- **Para frente, prospectivo:** identificação dos trabalhos recentes que citam os artigos origem, mostrando o estado da arte entre 2020 e 2025.

Os critérios de seleção foram: aderência direta à pergunta de pesquisa, impacto medido pelo número de citações para os clássicos, atualidade para os trabalhos recentes, e disponibilidade de texto completo. Ao final foram selecionadas 11 fontes, sendo 5 clássicas e 6 recentes, acima do mínimo de 10 fontes, com pelo menos 3 de cada tipo, pedido na atividade.

8 ARTIGOS ORIGEM

Artigo origem	Tipo	Por que é seed
Di Tommaso et al. (2015)	Clássico	Mede empiricamente o overhead do Docker em pipelines reais de bioinformática. É o trabalho mais citado diretamente sobre containers e eficiência de pipeline.
Felter et al. (2015)	Clássico	Comparação canônica entre VM e container, com cerca de 1.180 citações. Fundamenta a premissa de overhead desprezível dos containers.

Artigo origem	Tipo	Por que é seed
Pahl et al. (2019)	Clássico, revisão	Revisão de estado da arte sobre tecnologias de containers na nuvem. Bom hub de citações para expandir o mapa em direção a orquestração e ETL.

9 FONTES SELECIONADAS

Foram selecionadas 10 fontes no total, sendo 6 clássicas e 4 recentes, atendendo ao mínimo de 10 fontes com ao menos 3 de cada tipo exigido pela atividade.

9.1 Clássicos e mais citados

Referência	Ano	Veículo	Relevância para a pergunta
Felter et al.	2015	IEEE ISPASS	Comparação canônica entre VM e container em múltiplos benchmarks. Mostra overhead mínimo de containers, base empírica do argumento de eficiência.
Di Tommaso et al.	2015	PeerJ	Primeira medição empírica do overhead do Docker em pipelines reais, no domínio de bioinformática. Overhead desprezível para tarefas de maior duração.
Boettiger	2015	ACM SIGOPS OSDI	Estabelece o papel do Docker na reprodutibilidade científica, dimensão central da confiabilidade de pipelines em produção.
Morabito	2017	IEEE Access	Avaliação empírica de Docker e LXC em dispositivos IoT com recursos limitados. Quantifica o overhead de containers em cenários de coleta e processamento de dados em tempo real.
Zhang et al.	2018	IEEE CLOUD	Comparação empírica entre containers e VMs em workloads Spark de big data. Containers apresentam melhor escalabilidade e utilização de CPU e memória.
Pahl et al.	2019	IEEE TCC	Revisão sistemática das tecnologias de containers na nuvem. Consolida trade offs de performance e isolamento. Artigo origem do mapeamento.

9.2 Recentes, 2020 a 2025

Referência	Ano	Veículo	Relevância para a pergunta
Čilić et al.	2023	Sensors, MDPI	Avalia plataformas de orquestração de containers em edge computing em ambiente de rede real. Kubernetes apresenta potencial para escalonamento eficiente na borda da rede.
Park; Bahn	2023	Applied Sciences, MDPI	Quantifica o efeito do container em workloads de deep learning via rastreamento de chamadas de sistema. Solução proposta reduz latência de I/O em 82% na média.
Aqasizade et al.	2024	arXiv	Avalia empiricamente o overhead de containers rodando sobre máquinas virtuais em nuvem. Docker e Podman apresentam impacto negligível na maioria dos cenários.
Soltanmohammadi; Hikmet	2024	J. Data Anal. Inf. Proc.	ETL em saúde com PySpark e Docker sobre a base MIMIC III. Reporta ganhos de tempo de processamento, uso de recursos e escalabilidade.

10 COMPARAÇÃO DOS DADOS

Nessa seção, é realizada a comparação e a análise finais dos artigos selecionados. Para realizar esta análise, a ferramenta utilizada foi o NotebookLM, que permitiu reunir as evidências experimentais das fontes e organizar os achados em quatro eixos: virtualização e desempenho de sistema, engenharia de ETL e big data, orquestração de containers em ambientes de borda, e cargas de trabalho especializadas, como deep learning e genômica.

10.1 Comparação de tecnologias de virtualização: containers e VMs

A escolha entre máquinas virtuais (VMs) e containers é central na engenharia de infraestrutura de dados. Enquanto as VMs oferecem isolamento forte por meio do hipervisor, os containers compartilham o kernel do host, o que resulta em maior eficiência de recursos. A Tabela 1 reúne as principais métricas de desempenho de sistema observadas nos estudos.

Tabela 1: Métricas de eficiência e desempenho de sistema

Métrica	Containers (Docker/ Podman)	Máquinas virtuais (KVM/Xen)	Observações
Tempo de boot	~1,89 s	mais de 1.000 s em escala	VMs falham ao iniciar centenas de instâncias simultâneas.
Footprint de memória	~0,03 GB por instância	~0,23 GB por instância	O KVM exige de 3,6 a 4,6 vezes mais memória que os containers.
Overhead de CPU	desprezível (~2,67%)	baixo, porém superior aos containers	O Docker iguala ou supera o KVM em quase todos os testes.
Desempenho de I/O (disco)	próximo ao nativo, degradação de ~10%	superior no Xen via paravirtualização	O Xen (PVHVM) supera os containers em leitura e escrita em disco.

Fonte: elaborada pelo autor com base nas fontes analisadas (2026).

Os dados mostram que os containers inicializam em poucos segundos e consomem cerca de 0,03 GB por instância, contra 0,23 GB das VMs, o que representa um consumo de memória de 3,6 a 4,6 vezes menor. O overhead de CPU é praticamente desprezível. A principal vantagem das VMs aparece no desempenho de disco, no qual o Xen, por meio da paravirtualização, supera os containers em operações de leitura e escrita.

10.2 Eficiência em pipelines de ETL e big data

O processamento de grandes volumes de dados exige ferramentas escaláveis. A transição de um modelo de nó único, representado pelo Pandas, para um modelo distribuído, representado pelo PySpark, em ambientes containerizados, mostra ganhos expressivos de escala. A Tabela 2 apresenta os tempos de execução medidos sobre a base MIMIC III.

Tabela 2: Comparação de desempenho em engenharia de ETL (base MIMIC III)

Tamanho do dataset	Pandas (s)	PySpark (s)	PySpark em Docker (s)
0,25 GB	5	3	0,3
0,50 GB	15	5	0,5
1,00 GB	30	8	0,8
2,00 GB	55	8	0,8

Fonte: elaborada pelo autor com base em Soltanmohammadi e Hikmet (2024).

Enquanto o tempo de execução do Pandas cresce de forma linear com o tamanho do conjunto de dados, chegando a 55 segundos para 2 GB, o PySpark estabiliza em torno de 8 segundos. A versão containerizada do PySpark reduz ainda mais esse tempo, mantendo-se abaixo de 1 segundo em todos os volumes testados, o que confirma o ganho de desempenho da combinação entre processamento distribuído e containerização.

10.3 Orquestração em ambientes de borda

Em infraestruturas distribuídas, a escolha do orquestrador impacta diretamente a agilidade da rede e o consumo de recursos. O Kubernetes e suas versões leves, o K3s e o KubeEdge, são as soluções predominantes em dispositivos de recursos limitados. A Tabela 3 compara essas ferramentas.

Tabela 3: Comparação de orquestradores em dispositivos de recursos limitados (SBCs)

Ferramenta	Consumo de RAM (MB)	Tempo de startup do serviço (s)	Suporte a redes privadas
Kubernetes (K8s)	~50	1,799	Não (requer VPN externa)
K3s	~50	2,798	Sim (via tunnel proxy)
KubeEdge	~40	2,858	Sim (via WebSocket nuvem-borda)
ioFog	~240	34,537	Sim

Fonte: elaborada pelo autor com base em Čilić et al. (2023).

O Kubernetes padrão mostra-se eficiente em memória, com cerca de 50 MB de consumo, mas não oferece suporte nativo a redes privadas, dependendo de VPN externa. O K3s e o KubeEdge equilibram esse desempenho com flexibilidade de rede, viabilizando a inclusão de nós em redes privadas. O ioFog, ao contrário, apresenta consumo de memória muito superior e tempos de inicialização elevados, o que limita seu uso em cenários restritos.

10.4 Análise de gargalos em cargas de trabalho especializadas

A containerização não é isenta de desafios. Em aplicações de deep learning e em pipelines de curta duração, o gerenciamento de entrada e saída e a latência de inicialização

tornam-se pontos críticos. A Tabela 4 relaciona o impacto da containerização aos diferentes tipos de carga de trabalho.

Tabela 4: Impacto da containerização por tipo de carga de trabalho

Tipo de aplicação	Impacto do Docker	Causa principal do impacto	Solução proposta
Deep learning	Gargalo em escritas (write-back)	Sincronização frequente de páginas sujas (dirty pages)	Camada intermediária de memória não volátil (NVMe).
Genômica (tarefas longas)	Desprezível (0,1%)	Estabilidade do runtime isolado	Uso do Nextflow para transparência.
Genômica (tarefas curtas)	Alto (65% de lentidão)	Overhead de instanciar o container repetidamente	Agrupamento de tarefas (batching) ou processamento nativo.
Pesquisa científica	Positivo (reprodutibilidade)	Eliminação do inferno de dependências	Uso de Dockerfiles como documentação autoexecutável.

Fonte: elaborada pelo autor com base nas fontes analisadas (2026).

Os achados mostram que o impacto do Docker depende fortemente do tipo de tarefa. Em tarefas longas de genômica, o overhead é desprezível, próximo de 0,1%, mas em tarefas curtas ele pode chegar a 65% de lentidão, por causa do custo repetido de instanciar o container. No deep learning, o gargalo aparece nas operações de escrita, com a solução proposta no uso de camadas intermediárias de memória não volátil. Já na pesquisa científica, o efeito é positivo, pois a containerização elimina o chamado inferno de dependências e garante a reprodutibilidade dos resultados.

10.5 Síntese da comparação

De modo geral, a infraestrutura de dados moderna se beneficia da agilidade dos containers para escala horizontal e reprodutibilidade. Para sistemas de alta intensidade de escrita, como o deep learning, recomendam-se estratégias de cache em hardware. Em ambientes de borda, o K3s e o KubeEdge oferecem o melhor equilíbrio entre o desempenho do Kubernetes e a flexibilidade de rede exigida em campo. Os dados confirmam a resposta à pergunta de pesquisa: o uso de containers afeta positivamente a eficiência das pipelines de dados, com ganhos que se intensificam em ambientes distribuídos e de grande escala, ressalvados os casos específicos de tarefas de curtíssima duração.

REFERÊNCIAS

AQASIZADE, H.; ATAIE, E.; BASTAM, M. Experimental assessment of containers running on top of virtual machines. **arXiv**, 2024. arXiv:2401.07539. Disponível em: <https://arxiv.org/abs/2401.07539>. Acesso em: 22 jun. 2026.

BOETTIGER, C. An introduction to Docker for reproducible research, with examples from the R environment. **ACM SIGOPS Operating Systems Review**, v. 49, n. 1, p. 71-79, 2015.

DOI: 10.1145/2723872.2723882. Disponível em: <https://arxiv.org/abs/1410.0846>. Acesso em: 22 jun. 2026.

ČILIĆ, I. et al. Performance evaluation of container orchestration tools in edge computing environments. **Sensors**, v. 23, n. 8, p. 4008, 2023. DOI: 10.3390/s23084008. Disponível em: <https://www.mdpi.com/1424-8220/23/8/4008>. Acesso em: 22 jun. 2026.

DI TOMMASO, P. et al. The impact of Docker containers on the performance of genomic pipelines. **PeerJ**, v. 3, e1273, 2015. DOI: 10.7717/peerj.1273. Disponível em: <https://peerj.com/articles/1273/>. Acesso em: 22 jun. 2026.

FELTER, W. et al. An updated performance comparison of virtual machines and Linux containers. In: IEEE INTERNATIONAL SYMPOSIUM ON PERFORMANCE ANALYSIS OF SYSTEMS AND SOFTWARE (ISPASS), 2015. **Anais [...]**. [S. l.]: IEEE, 2015. p. 171-172. DOI: 10.1109/ISPASS.2015.7095802.

MORABITO, R. Virtualization on Internet of Things edge devices with container technologies: a performance evaluation. **IEEE Access**, v. 5, p. 8835-8850, 2017. DOI: 10.1109/ACCESS.2017.2718650. Disponível em: <https://ieeexplore.ieee.org/document/7946339>. Acesso em: 22 jun. 2026.

PAHL, C. et al. Cloud container technologies: a state of the art review. **IEEE Transactions on Cloud Computing**, v. 7, n. 3, p. 677-692, 2019. DOI: 10.1109/TCC.2017.2702586. Disponível em: <https://pooyanjamshidi.github.io/resources/papers/cloud-containers-tcc.pdf>. Acesso em: 22 jun. 2026.

PARK, S.; BAHN, H. Performance analysis of container effect in deep learning workloads and implications. **Applied Sciences**, v. 13, n. 21, p. 11654, 2023. DOI: 10.3390/app132111654. Disponível em: <https://www.mdpi.com/2076-3417/13/21/11654>. Acesso em: 22 jun. 2026.

SOLTANMOHAMMADI, E.; HIKMET, N. Optimizing healthcare big data processing with containerized PySpark and parallel computing: a study on ETL pipeline efficiency. **Journal of Data Analysis and Information Processing**, v. 12, n. 4, p. 544-565, 2024. DOI: 10.4236/jdaip.2024.124029. Disponível em: <https://www.scirp.org/journal/paperinformation?paperid=136659>. Acesso em: 22 jun. 2026.

ZHANG, Q. et al. A comparative study of containers and virtual machines in big data environment. In: IEEE INTERNATIONAL CONFERENCE ON CLOUD COMPUTING (CLOUD), 2018. **Anais [...]**. [S. l.]: IEEE, 2018. p. 178-185. DOI: 10.1109/CLOUD.2018.00030. Disponível em: <https://arxiv.org/abs/1807.01842>. Acesso em: 22 jun. 2026.